Tim Pengajar IF2150

IF2150 – Rekayasa Perangkat Lunak

Software Quality

SEMESTER I TAHUN AJARAN 2025/2026

KNOWLEDGE & SOFTWARE ENGINEERING

Objective

- Students understand the goal of software quality
- Students know various aspects related to software quality
  - Dilemma
  - Quality attribute
- Students know various approaches to ensure software quality
- Quality Assurance vs Quality Check

Software quality

- Quality, simplistically, means that a **product should meet its specification**.
- This is problematical for software systems
  - Software specifications are usually incomplete and often inconsistent.
  - There is a tension between customer quality requirements (efficiency, reliability, etc.) and developer quality requirements (maintainability, reusability, etc.);
  - Some quality requirements are difficult to specify in an unambiguous way;
- The focus may be '**fitness for purpose**' rather than specification conformance.

Software Quality

- In 2005, *ComputerWorld* [Hil05] lamented that
  - "bad software plagues nearly every organization that uses computers, causing lost work hours during computer downtime, lost or corrupted data, missed sales opportunities, high IT support and maintenance costs, and low customer satisfaction.
- A year later, *InfoWorld* [Fos06] wrote about the
  - "the sorry state of software quality" reporting that the quality problem had not gotten any better.
- Today, software quality remains an issue, but who is to blame?
  - Customers blame developers, arguing that sloppy practices lead to low-quality software.
  - Developers blame customers (and other stakeholders), arguing that irrational delivery dates and a continuing stream of changes force them to deliver software before it has been fully validated.

Quality

- The *American Heritage Dictionary* defines *quality* as
  - "a characteristic or attribute of something."
- For software, two kinds of quality may be encountered:
  - Quality of design encompasses requirements, specifications, and the design of the system.
  - Quality of conformance is an issue focused primarily on implementation.
  - User satisfaction = compliant product + good quality + delivery within budget and schedule

Quality—A Philosophical View

- Robert Persig [Per74] commented on the thing we call quality:
  - Quality . . . you know what it is, yet you don't know what it is. But that's self-contradictory. But some things are better than others, that is, they have more quality. But when you try to say what the quality is, apart from the things that have it, it all goes poof! There's nothing to talk about. But if you can't say what Quality is, how do you know what it is, or how do you know that it even exists? If no one knows what it is, then for all practical purposes it doesn't exist at all. But for all practical purposes it really does exist. What else are the grades based on? Why else would people pay fortunes for some things and throw others in the trash pile? Obviously some things are better than others . . . but what's the betterness? . . . So round and round you go, spinning mental wheels and nowhere finding anyplace to get traction. What the hell is Quality? What is it?

Quality—A Pragmatic View

- The ***transcendental view*** argues (like Persig) that quality is something that you immediately recognize but cannot explicitly define.
- The ***user view*** sees quality in terms of an end-user's specific goals. If a product meets those goals, it exhibits quality.
- The ***manufacturer's view*** defines quality in terms of the original specification of the product. If the product conforms to the spec, it exhibits quality.
- The ***product view*** suggests that quality can be tied to inherent characteristics (e.g., functions and features) of a product.
- Finally, the ***value-based view*** measures quality based on how much a customer is willing to pay for a product. In reality, quality encompasses all of these views and more.

Comment on Quality

- Phil Crosby once said:
  - The problem of quality management is not what people don't know about it. The problem is what they think they do know . . .
  - *Everybody is for it.* (Under certain conditions, of course.)
  - *Everyone feels they understand it.* (Even though they wouldn't want to explain it.)
  - *Everyone thinks execution is only a matter of following natural inclinations.* (After all, we do get along somehow.)
  - *And, of course, most people feel that problems in these areas are caused by other people.* (If only they would take the time to do things right.)

Software Quality

- Software quality can be defined as:
  - *An effective software process applied in a manner that creates a* ***useful product*** *that provides measurable value for those who produce it and those who use it.* (adapted from [Bes04])

Useful Product

- A *useful product* delivers the content, functions, and features that the end-user desires
  - But as important, it delivers these assets in a reliable, error free way.
- A useful product always satisfies those requirements that have been explicitly stated by stakeholders.
  - In addition, it satisfies a set of implicit requirements (e.g., ease of use) that are expected of all high quality software.

Fitness for purpose

- Has the software been properly tested?
- Is the software sufficiently dependable to be put into use?
- Is the performance of the software acceptable for normal use?
- Is the software usable?
- Is the software well-structured and understandable?
- Have programming and documentation standards been followed in the development process?

Adding Value

- By *adding value for both the **producer** and **user*** of a software product, high quality software provides **benefits** for the **software organization** and the **end-user community**.
- The software organization gains added value because high quality software requires **less maintenance effort**, fewer bug fixes, and reduced customer support.
- The user community gains added value because the **application provides a useful capability** in a way that expedites some business process.
- The end result is:
  - (1) greater software product revenue,
  - (2) better profitability when an application supports a business process, and/or
  - (3) improved availability of information that is crucial for the business.

Quality Dimensions

- David Garvin [Gar87]:
  - **Performance Quality.** Does the software deliver all content, functions, and features that are specified as part of the requirements model in a way that provides value to the end-user?
  - **Feature quality.** Does the software provide features that surprise and delight first-time end-users?
  - **Reliability.** Does the software deliver all features and capability without failure? Is it available when it is needed? Does it deliver functionality that is error free?
  - **Conformance.** Does the software conform to local and external software standards that are relevant to the application? Does it conform to de facto design and coding conventions? For example, does the user interface conform to accepted design rules for menu selection or data input?

Quality Dimensions

- **Durability.** Can the software be maintained (changed) or corrected (debugged) without the inadvertent generation of unintended side effects? Will changes cause the error rate or reliability to degrade with time?
- **Serviceability.** Can the software be maintained (changed) or corrected (debugged) in an acceptably short time period. Can support staff acquire all information they need to make changes or correct defects?
- **Aesthetics.** Most of us would agree that an aesthetic entity has a certain elegance, a unique flow, and an obvious "presence" that are hard to quantify but evident nonetheless.
- **Perception.** In some situations, you have a set of prejudices that will influence your perception of quality.

Software Quality – ISO 9126

- **Functionality** - "A set of attributes that bear on the existence of a set of **functions** and their specified **properties**. The functions are those that satisfy stated or implied needs."
- **Reliability** - "A set of attributes that bear on the capability of software to **maintain its level of performance** under stated conditions for a stated period of time."
- **Usability** - "A set of attributes that bear on the **effort needed for use**, and on the individual assessment of such use, by a stated or implied set of users."
- **Efficiency** - "A set of attributes that bear on the relationship between the level of performance of the software and the amount of **resources used**, under stated conditions."
- **Maintainability** - "A set of attributes that bear on the **effort needed** to make specified **modifications**.
- "**Portability** - "A set of attributes that bear on the ability of software to be **transferred from one environment to another**."

KNOWLEDGE & SOFTWARE ENGINEERING

Software Quality – ISO 9126

- Functionality
  - Suitability
  - Accuracy
  - Interoperability
  - Security
  - Functionality compliance
- Reliability
  - Maturity
  - Fault tolerance
  - Recoverability
  - Reliability compliance
- Usability
  - Understandability
  - Learnability
  - Operability
  - Attractiveness
  - Usability compliance
- Efficiency
  - Time behavior
  - Resource utilization
  - Efficiency compliance
- Maintainability
  - Analyzability
  - Changeability
  - Stability
  - Testability
  - Maintainability compliance
- Portability
  - Adaptability
  - Installability
  - Co-existence
  - Replaceability
  - Portability compliance

Non-functional characteristics

- The **subjective quality** of a software system is largely based on its **non-functional characteristics**.
  - This reflects practical user experience – if the software's functionality is not what is expected, then users will often just work around this and find other ways to do what they want to do.
  - However, if the software is unreliable or too slow, then it is practically impossible for them to achieve their goals.

Software quality attributes

Boehm, et al. (1978) suggested that there were 15 important software quality attributes

Safety	Understandability	Portability
Security	Testability	Usability
Reliability	Adaptability	Reusability
Resilience	Modularity	Efficiency
Robustness	Complexity	Learnability

Quality conflicts

- It is **not possible** for any system to be **optimized for all** of these attributes – for example, improving robustness may lead to loss of performance.
- The quality plan should therefore **define the most important quality** attributes for the software that is being developed.
- The plan should also **include** a definition of the **quality assessment process**, an agreed way of assessing whether some quality, such as maintainability or robustness, is present in the product.

The Software Quality Dilemma

- If you produce a software system that has terrible quality, you lose because no one will want to buy it.
- If on the other hand you spend infinite time, extremely large effort, and huge sums of money to build the absolutely perfect piece of software, then it's going to take so long to complete and it will be so expensive to produce that you'll be out of business anyway.
- Either you missed the market window, or you simply exhausted all your resources.
- So people in industry try to get to that magical **middle ground** where the product is **good enough** not to be rejected right away, such as during evaluation, but also not the object of so much perfectionism and so much work that it would take too long or cost too much to complete. [Ven03]

"Good Enough" Software

- **Good enough software** delivers high quality functions and features that end-users desire, but at the same time it delivers other more obscure or specialized functions and features that contain known bugs.

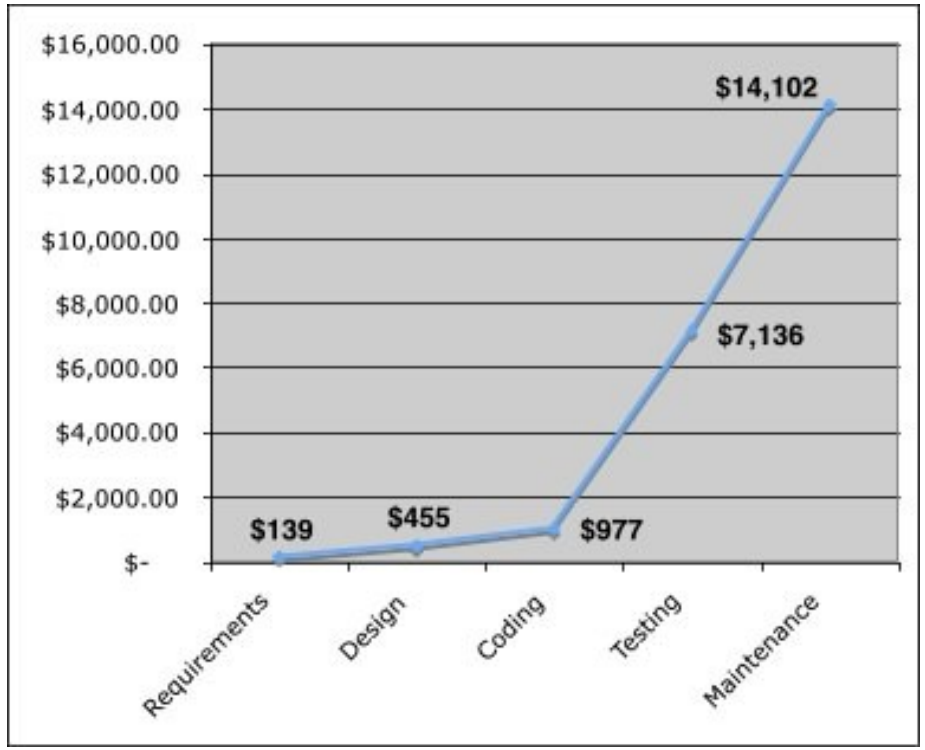
- Arguments *against* "good enough."
  - It is true that "good enough" may work in some application domains and for a few major software companies. After all, if a company has a large marketing budget and can convince enough people to buy version 1.0, it has succeeded in locking them in.
  - If you work for a small company be wary of this philosophy. If you deliver a "good enough" (buggy) product, you risk permanent damage to your company's reputation.
  - You may never get a chance to deliver version 2.0 because bad buzz may cause your sales to plummet and your company to fold.
  - If you work in certain application domains e.g., real time embedded software, application software that is integrated with hardware can be negligent and open your company to expensive litigation.

Cost of Quality

- ***Prevention costs*** include
  - quality planning
  - formal technical reviews
  - test equipment
  - Training
- ***Internal failure costs*** include
  - rework
  - repair
  - failure mode analysis
- ***External failure costs*** are
  - complaint resolution
  - product return and replacement
  - help line support
  - warranty work

Cost

- The relative costs to find and repair an error or defect increase dramatically as we go from prevention to detection to internal failure to external failure costs.



Quality and Risk

- *"People bet their jobs, their comforts, their safety, their entertainment, their decisions, and their very lives on computer software. It better be right."* SEPA, Chapter 1
- Example:
  - *Throughout the month of November, 2000 at a hospital in Panama, 28 patients received massive overdoses of gamma rays during treatment for a variety of cancers. In the months that followed, five of these patients died from radiation poisoning and 15 others developed serious complications. What caused this tragedy? A software package, developed by a U.S. company, was modified by hospital technicians to compute modified doses of radiation for each patient.*

Negligence and Liability

- The story is all too common. A governmental or corporate entity hires a major software developer or consulting company to analyze requirements and then design and construct a software-based "system" to support some major activity.
  - The system might support a major corporate function (e.g., pension management) or some governmental function (e.g., healthcare administration or homeland security).
- Work begins with the best of intentions on both sides, but by the time the system is delivered, things have gone bad.
- The system is late, fails to deliver desired features and functions, is error-prone, and does not meet with customer approval.
- Litigation ensues.

Quality and Security

- Gary McGraw comments [Wil05]:
- "Software security relates entirely and completely to quality. You must think about security, reliability, availability, dependability—at the beginning, in the design, architecture, test, and coding phases, all through the software life cycle [process]. Even people aware of the software security problem have focused on late life-cycle stuff. The earlier you find the software problem, the better. And there are two kinds of software problems. One is bugs, which are implementation problems. The other is software flaws—architectural problems in the design. *People pay too much attention to bugs and not enough on flaws.*"

Achieving Software Quality

- Critical success factors:
  - Software Engineering Methods
  - Project Management Techniques
  - Quality Control
  - Quality Assurance

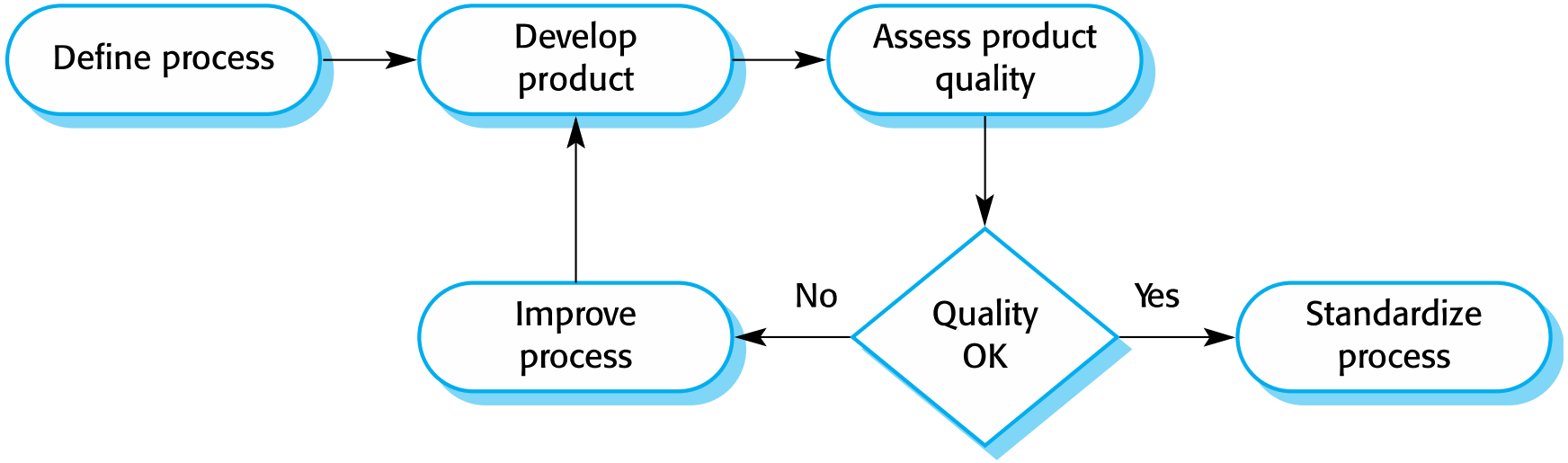
Effective Software Process

- An **effective software process** establishes the infrastructure that supports any effort at building a high quality software product.
- The **management aspects** of process create the checks and balances that help avoid project chaos—a key contributor to poor quality.
- **Software engineering practices** allow the developer to analyze the problem and design a solid solution—both critical to building high quality software.
- Finally, **umbrella activities** such as change management and technical reviews have as much to do with quality as any other part of software engineering practice.

Process and product quality

- The quality of a developed product is influenced by the quality of the production process.
- This is important in software development as some product quality attributes are hard to assess.
- However, there is a very complex and poorly understood relationship between software processes and product quality.
  - The application of **individual skills** and **experience** is particularly important in software development;
  - External factors such as the novelty of an application or the need for an accelerated development schedule may impair product quality.

Process-based quality



Quality culture

- Quality managers should aim to develop a ‘quality culture’ where **everyone** responsible for software development is **committed** to achieving a high level of product quality.
- They should encourage teams to take responsibility for the quality of their work and to develop new approaches to quality improvement.
- They should support people who are interested in the intangible aspects of quality and encourage professional behavior in all team members.

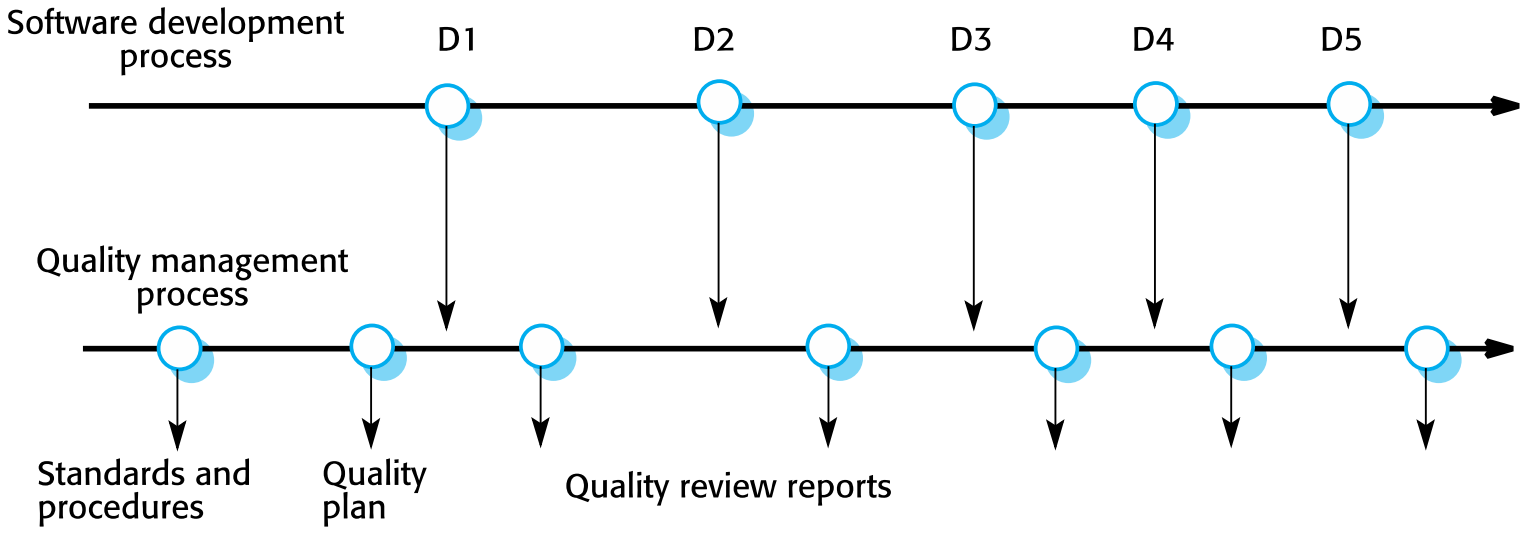
Software quality management

- Concerned with **ensuring** that the required level of quality is **achieved** in a software product.
- Three principal concerns:
  - At the organizational level, quality management is concerned with **establishing a framework** of organizational processes and standards that will lead to high-quality software.
  - At the project level, quality management involves the **application of specific quality processes** and checking that these planned processes have been followed.
  - At the project level, quality management is also concerned with **establishing a quality plan** for a project. The quality plan should set out the quality goals for the project and define what processes and standards are to be used.

Quality management activities

- Quality management provides an **independent check** on the software development process.
- The quality management process **checks the project deliverables** to ensure that they are consistent with organizational standards and goals
- The quality team should be independent from the development team so that they can take an **objective view of the software**. This allows them to report on software quality without being influenced by software development issues.

Quality management and software development



Software Quality Assurance vs Control

Quality Assurance

- Preventing defect and proactive
- Ensure the approaches, techniques, methods, and processes are designed for the projects are implemented correctly
- Monitor & verify the processes are followed

Quality Control

- Identifying/detecting defect and reactive
- Ensure the deliverables meet the requirements
- Validate the deliverables

Software Quality Assurance

Elements of SQA

- Standards
- Reviews and Audits
- Testing
- Error/defect collection and analysis
- Change management
- Education
- Vendor management
- Security management
- Safety
- Risk management

Role of the SQA Group

- **Prepares an SQA plan for a project.**
  - The plan identifies
    - evaluations to be performed
    - audits and reviews to be performed
    - standards that are applicable to the project
    - procedures for error reporting and tracking
    - documents to be produced by the SQA group
    - amount of feedback provided to the software project team
- **Participates in the development of the project's software process description.**
  - The SQA group **reviews the process description** for compliance with organizational policy, internal software standards, externally imposed standards (e.g., ISO-9001), and other parts of the software project plan.

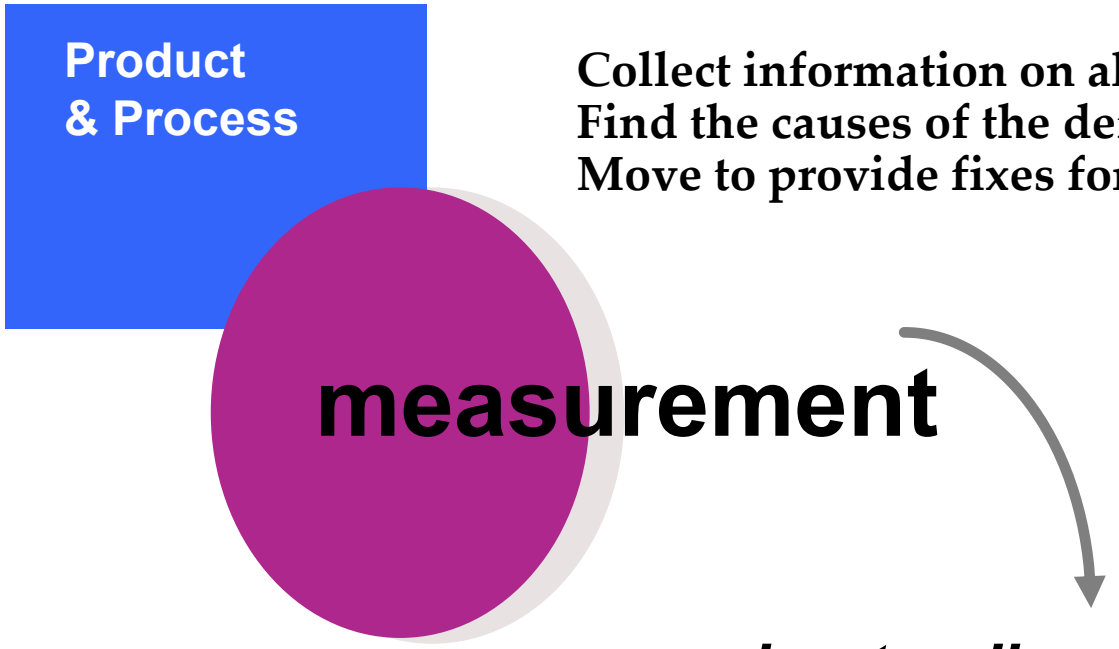
Role of the SQA Group (2)

- **Reviews software engineering activities to verify compliance with the defined software process.**
  - identifies, documents, and tracks deviations from the process and verifies that corrections have been made.
- **Audits designated software work products to verify compliance with those defined as part of the software process.**
  - reviews selected work products; identifies, documents, and tracks deviations; verifies that corrections have been made
  - periodically reports the results of its work to the project manager.
- **Ensures that deviations in software work and work products are documented and handled according to a documented procedure.**
- **Records any noncompliance and reports to senior management.**
  - Noncompliance items are tracked until they are resolved.

SQA Goals

- **Requirements quality**. The correctness, completeness, and consistency of the requirements model will have a strong influence on the quality of all work products that follow.
- **Design quality**. Every element of the design model should be assessed by the software team to ensure that it exhibits high quality and that the design itself conforms to requirements.
- **Code quality**. Source code and related work products (e.g., other descriptive information) must conform to local coding standards and exhibit characteristics that will facilitate maintainability.
- **Quality control effectiveness**. A software team should apply limited resources in a way that has the highest likelihood of achieving a high-quality result.

Statistical SQA



Collect information on all defects
Find the causes of the defects
Move to provide fixes for the process

... an understanding of how to improve quality ...

Statistical SQA

- **Information** about software errors and defects is **collected** and **categorized**.
- An attempt is made to **trace** each error and defect **to its underlying cause** (e.g., non-conformance to specifications, design error, violation of standards, poor communication with the customer).
- Using the **Pareto principle** (80 percent of the defects can be traced to 20 percent of all possible causes), isolate the 20 percent (the *vital few*).
- Once the vital few causes have been identified, move to correct the problems that have caused the errors and defects.

Six-Sigma for Software Engineering

- The term "six sigma" is derived from six standard deviations—3.4 instances (defects) per million occurrences—implying an extremely high-quality standard.
- The Six Sigma methodology defines five core steps:
  - ***Define*** customer requirements and deliverables and project goals via well-defined methods of customer communication
  - ***Measure*** the existing process and its output to determine current quality performance (collect defect metrics)
  - ***Analyze*** defect metrics and determine the vital few causes.
  - ***Improve*** the process by eliminating the root causes of defects.
  - ***Control*** the process to ensure that future work does not reintroduce the causes of defects.

Software Reliability

- A simple measure of reliability is *mean-time-between-failure* (MTBF), where

$$MTBF = MTTF + MTTR$$

- The acronyms MTTF and MTTR are *mean-time-to-failure* and *mean-time-to-repair*, respectively.
- *Software availability* is the probability that a program is operating according to requirements at a given point in time and is defined as

$$Availability = [MTTF/(MTTF + MTTR)] \times 100\%$$

Software Safety

- ***Software safety*** is a software quality assurance activity that focuses on the identification and assessment of potential hazards that may affect software negatively and cause an entire system to fail.
- If hazards can be identified early in the software process, software design features can be specified that will either eliminate or control potential hazards.

ISO 9001:2000 Standard

- ISO 9001:2000 is the quality assurance standard that applies to software engineering.
- The standard contains **20 requirements** that must be present for an effective quality assurance system.
- The requirements delineated by ISO 9001:2000 address topics such as
  - management responsibility, quality system, contract review, design control, document and data control, product identification and traceability, process control, inspection and testing, corrective and preventive action, control of quality records, internal quality audits, training, servicing, and statistical techniques.

Credits

- The slides are adapted from
  - *Software Engineering: A Practitioner's Approach, 7/e* **by Roger S. Pressman**
  - Software Engineering, 10th **by Ian Sommerville**